University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title:

Relational Database Design, Implementation, and Analytics

Domain: - Hospital Management System Database

Programme Title: MSc Data Analytics

Name: AYUSH RAVINDRA VINCHURKAR

Professor: Prof. Mohammed Alimam

Word Limit: 3300 Words

Submission Date: 16-06-2026

Year: 2025–2026

CONTENTS

Table of Contents

Introduction

Task 1: Basic Database Design

1. Chosen Domain
2. SQL Table Creation
3. ERD Design
4. Basic Normalization to 3NF

Task 2: Insert Sample Data and Create Views

1. Sample Data Insertion
2. View 1: Patient Bill Summary
3. View 2: Monthly Appointment Trend

Task 3: Essential SQL Querying and Analytics

1. Query 1: JOIN and GROUP BY
2. Query 2: CASE Classification
3. Query 3: Window Function

Task 4: CAP Theorem

Task 5: Query Performance Improvement

Conclusion

Bibliography

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters): AYUSH RAVINDRA VINCHURKAR

Date: 16/06/2026

INTRODUCTION

A database management system is applied for the storing, organizing, managing, and analyzing structured data. Databases are very crucial in the healthcare industry due to the fact that hospital stores a huge amount of information regarding patients, doctors, departments, appointment, treatments, and bills. In other words, lack of organization on the side of data might lead to duplication, inconsistency, slow reporting, and poor decision making.

This report provides the design and implementation of a relational database for the Hospital Management System using MySQL database software. Database design and implementation include the following: database design, SQL tables, database ERD structure, database normalization, SQL table data population, view creating and manipulating, analytical SQL query creation, application of CAP theorem and optimization of queries.

Hospitals' database management system domain was chosen for this project because there are several entities within this domain, and they are realistically related. For instance, one department may contain many doctors, one doctor handles several appointments, one patient participates in many appointments, and one patient undergoes many treatments. Therefore, there are many real-world relations which allow us to implement relational database concepts such as primary key, foreign key, many-to-many relationship, view, join, group by, case classification, and window function.

Task 1: Basic Database Design

1. **Chosen Domain**

   The selected domain for this database project is a Hospital Management System. The hospital collects data on the patients, physicians, departments, appointments, treatment, and bills. The patients use the hospital management system to make appointments and get treatment. The doctors, nurses, and hospital personnel utilize the database to organize appointments, treat patients, and bill the patients. The management can analyze the collected data in order to determine the most popular doctors, trends in the appointment patterns, and high medical charges by patients.

The database is designed using seven tables:

1. Departments
2. Doctors
3. Patients
4. Appointments
5. Treatments
6. PatientTreatments
7. Bills

   This design uses primary keys to uniquely identify each record and foreign keys to connect related tables.

2. **SQL Table Creation**

   The following SQL script creates the Hospital Management System database and all required tables.

```sql
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL,
    Location VARCHAR(100),
    Phone VARCHAR(15)
);

CREATE TABLE Doctors (
    DoctorID INT PRIMARY KEY,
    DoctorName VARCHAR(100) NOT NULL,
    Specialization VARCHAR(100),
    Phone VARCHAR(15),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);

CREATE TABLE Patients (
    PatientID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Gender VARCHAR(10),
    DateOfBirth DATE,
    Phone VARCHAR(15)
);

CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY,
    PatientID INT,
    DoctorID INT,
    AppointmentDate DATE,
    AppointmentTime TIME,
    Status VARCHAR(30),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);

CREATE TABLE Treatments (
    TreatmentID INT PRIMARY KEY,
    TreatmentName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    Cost DECIMAL(10,2)
);

CREATE TABLE PatientTreatments (
    PatientTreatmentID INT PRIMARY KEY,
    PatientID INT,
```

```
    TreatmentID INT,
    TreatmentDate DATE,
    Notes VARCHAR(255),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID)
);

CREATE TABLE Bills (
    BillID INT PRIMARY KEY,
    PatientID INT,
    BillDate DATE,
    Amount DECIMAL(10,2),
    PaymentStatus VARCHAR(30),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
```

Details of the departments like name, address, and telephone number can be found in the Departments table. The doctors' details are found in the Doctors table, where every doctor belongs to a particular department. Details related to patients' demography can be found in the Patients table. Appointment schedule of doctors for various patients can be found in the Appointments table. Details of the treatment procedures along with their description and costs can be found in the Treatments table.

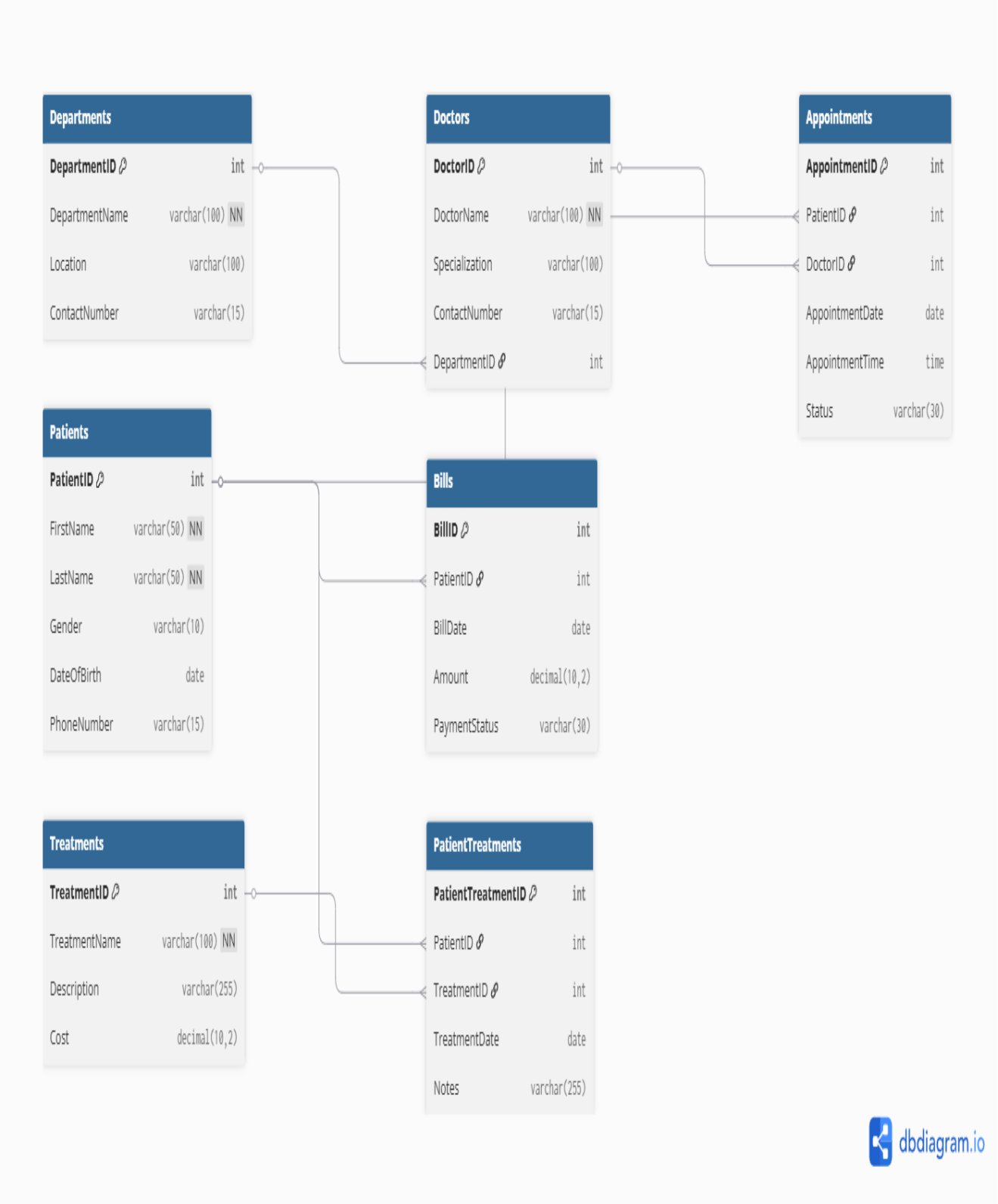
3. ERD Design

The Entity Relationship Diagram shows the structure of the database, including tables, primary keys, foreign keys, and relationships.

The main relationships are:

- One department can have many doctors.
- One doctor can have many appointments.
- One patient can have many appointments.
- One patient can receive many treatments.
- One treatment can be given to many patients.
- One patient can have many bills.

The connection between patients and treatment is a many to many relationships. The problem is overcome by having a table called PatientTreatments. In the absence of this linking table, we would either have duplicate columns in the patient table for treatments or in the treatment table for patients.


Departments
DepartmentID int
DepartmentName varchar(100) NN
Location varchar(100)
ContactNumber varchar(15)
Doctors
DoctorID int
DoctorName varchar(100) NN
Specialization varchar(100)
ContactNumber varchar(15)
DepartmentID int
Appointments
AppointmentID int
PatientID int
DoctorID int
AppointmentDate date
AppointmentTime time
Status varchar(30)
Patients
PatientID int
FirstName varchar(50) NN
LastName varchar(50) NN
Gender varchar(10)
DateOfBirth date
PhoneNumber varchar(15)
Bills
BillID int
PatientID int
BillDate date
Amount decimal(10,2)
PaymentStatus varchar(30)
Treatments
TreatmentID int
TreatmentName varchar(100) NN
Description varchar(255)
Cost decimal(10,2)
PatientTreatments
PatientTreatmentID int
PatientID int
TreatmentID int
TreatmentDate date
Notes varchar(255)
dbdiagram.io

4. **Basic Normalization to 3NF:**

   Before normalization, doctor and department information could be stored in one single table. This design is not efficient because department details are repeated for every doctor.

DoctorID	DoctorName	Specialization	DoctorContact	DepartmentName	DepartmentLocation	DepartmentContact
101	Dr. John Smith	Cardiologist	9876543210	Cardiology	Building A	1111111111
102	Dr. Sarah Brown	Neurologist	9876543211	Neurology	Building B	2222222222
103	Dr. Michael Lee	Orthopedic Surgeon	9876543212	Orthopedics	Building C	3333333333
104	Dr. Emma Wilson	Pediatrician	9876543213	Pediatrics	Building D	4444444444

This is a repetition of data in regard to the departments. If a number of doctors belonged to the same department, then the name of the department, the address of the department, and other related contacts would have been repeated, causing data redundancy and inconsistencies. For instance, if the Cardiology department changed location, it meant repeating all that data regarding the department.

To improve the design, the table was split into two separate tables: Departments and Doctors.

After Normalization: Departments Table

DepartmentID	DepartmentName	Location	ContactNumber
1	Cardiology	Building A	1111111111
2	Neurology	Building B	2222222222
3	Orthopedics	Building C	3333333333
4	Pediatrics	Building D	4444444444

After Normalization: Doctors Table

DoctorID	DoctorName	Specialization	ContactNumber	DepartmentID
101	Dr. John Smith	Cardiologist	9876543210	1
102	Dr. Sarah Brown	Neurologist	9876543211	2
103	Dr. Michael Lee	Orthopedic Surgeon	9876543212	3
104	Dr. Emma Wilson	Pediatrician	9876543213	4

The improved schema is designed based on Third Normal Form since the attributes associated with the department are entirely dependent on DepartmentID and not on the DoctorID. Doctors store all information related to doctors, while Departments store all the attributes that relate to departments.

Task 2: Insert Sample Data and Create Views

1. Sample Data Insertion

The following SQL inserts sample records into each table.

```
71 •  INSERT INTO Departments
72    (DepartmentID, DepartmentName, Location, Phone)
73    VALUES
74    (1, 'Cardiology', 'Building A', '1111111111'),
75    (2, 'Neurology', 'Building B', '2222222222'),
76    (3, 'Orthopedics', 'Building C', '3333333333'),
77    (4, 'Pediatrics', 'Building D', '4444444444');
78
79 •  INSERT INTO Doctors
80    (DoctorID, DoctorName, Specialization, Phone, DepartmentID)
81    VALUES
82    (101, 'Dr. John Smith', 'Cardiologist', '9876543210', 1),
83    (102, 'Dr. Sarah Brown', 'Neurologist', '9876543211', 2),
84    (103, 'Dr. Michael Lee', 'Orthopedic Surgeon', '9876543212', 3),
85    (104, 'Dr. Emma Wilson', 'Pediatrician', '9876543213', 4);
86
87 •  INSERT INTO Patients
88    (PatientID, FirstName, LastName, Gender, DateOfBirth, Phone)
89    VALUES
90    (201, 'James', 'Anderson', 'Male', '1990-05-15', '9000000001'),
91    (202, 'Sophia', 'Taylor', 'Female', '1995-08-20', '9000000002'),
92    (203, 'Daniel', 'Thomas', 'Male', '1988-11-10', '9000000003'),
93    (204, 'Olivia', 'Martin', 'Female', '1992-03-25', '9000000004'),
94    (205, 'William', 'White', 'Male', '1998-07-18', '9000000005');
```

```sql

INSERT INTO Appointments
(AppointmentID, PatientID, DoctorID, AppointmentDate, AppointmentTime, Status)
VALUES
(301, 201, 101, '2026-05-01', '09:00:00', 'Completed'),
(302, 202, 102, '2026-05-03', '10:00:00', 'Completed'),
(303, 203, 103, '2026-05-05', '11:00:00', 'Completed'),
(304, 204, 101, '2026-06-07', '12:00:00', 'Pending'),
(305, 205, 104, '2026-06-10', '14:00:00', 'Completed');

INSERT INTO Treatments
(TreatmentID, TreatmentName, Description, Cost)
VALUES
(401, 'Heart Checkup', 'Cardiac examination and consultation', 500.00),
(402, 'Brain MRI', 'Neurological scan for brain diagnosis', 1200.00),
(403, 'Bone Surgery', 'Orthopedic surgical procedure', 5000.00),
(404, 'ECG Test', 'Heart rhythm monitoring test', 300.00),
(405, 'Child Health Checkup', 'General pediatric health examination', 250.00);

INSERT INTO PatientTreatments
(PatientTreatmentID, PatientID, TreatmentID, TreatmentDate, Notes)
VALUES
(501, 201, 401, '2026-05-01', 'Routine cardiac checkup completed'),
(502, 202, 402, '2026-05-03', 'MRI scan completed'),
(503, 203, 403, '2026-05-05', 'Surgery completed successfully'),
(504, 204, 404, '2026-06-07', 'ECG test performed'),
(505, 205, 405, '2026-06-10', 'General pediatric screening completed');

```

```sql
INSERT INTO Bills
(BillID, PatientID, BillDate, Amount, PaymentStatus)
VALUES
(601, 201, '2026-05-01', 500.00, 'Paid'),
(602, 202, '2026-05-03', 1200.00, 'Paid'),
(603, 203, '2026-05-05', 5000.00, 'Pending'),
(604, 204, '2026-06-07', 300.00, 'Paid'),
(605, 205, '2026-06-10', 250.00, 'Paid');

SELECT * FROM Departments;
SELECT * FROM Doctors;
SELECT * FROM Patients;
SELECT * FROM Appointments;
SELECT * FROM Treatments;
SELECT * FROM PatientTreatments;
SELECT * FROM Bills;
```

SELECT * FROM Departments;

DepartmentID	DepartmentName	Location	Phone
1	Cardiology	Building A	1111111111
2	Neurology	Building B	2222222222
3	Orthopedics	Building C	3333333333
4	Pediatrics	Building D	4444444444
NULL	NULL	NULL	NULL

SELECT * FROM Doctors;

DoctorID	DoctorName	Specialization	Phone	DepartmentID
101	Dr. John Smith	Cardiologist	9876543210	1
102	Dr. Sarah Brown	Neurologist	9876543211	2
103	Dr. Michael Lee	Orthopedic Surgeon	9876543212	3
104	Dr. Emma Wilson	Pediatrician	9876543213	4
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Patients;

PatientID	FirstName	LastName	Gender	DateOfBirth	Phone
201	James	Anderson	Male	1990-05-15	9000000001
202	Sophia	Taylor	Female	1995-08-20	9000000002
203	Daniel	Thomas	Male	1988-11-10	9000000003
204	Olivia	Martin	Female	1992-03-25	9000000004
205	William	White	Male	1998-07-18	9000000005
NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM Appointments;

AppointmentID	PatientID	DoctorID	AppointmentDate	AppointmentTime	Status
301	201	101	2026-05-01	09:00:00	Completed
302	202	102	2026-05-03	10:00:00	Completed
303	203	103	2026-05-05	11:00:00	Completed
304	204	101	2026-06-07	12:00:00	Pending
305	205	104	2026-06-10	14:00:00	Completed
NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM Treatments;

TreatmentID	TreatmentName	Description	Cost
401	Heart Checkup	Cardiac examination and consultation	500.00
402	Brain MRI	Neurological scan for brain diagnosis	1200.00
403	Bone Surgery	Orthopedic surgical procedure	5000.00
404	ECG Test	Heart rhythm monitoring test	300.00
405	Child Health Checkup	General pediatric health examination	250.00
NULL	NULL	NULL	NULL

SELECT * FROM PatientTreatments;

PatientTreatmentID	PatientID	TreatmentID	TreatmentDate	Notes
501	201	401	2026-05-01	Routine cardiac checkup completed
502	202	402	2026-05-03	MRI scan completed
503	203	403	2026-05-05	Surgery completed successfully
504	204	404	2026-06-07	ECG test performed
505	205	405	2026-06-10	General pediatric screening completed
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Bills;

BillID	PatientID	BillDate	Amount	PaymentStatus
601	201	2026-05-01	500.00	Paid
602	202	2026-05-03	1200.00	Paid
603	203	2026-05-05	5000.00	Pending
604	204	2026-06-07	300.00	Paid
605	205	2026-06-10	250.00	Paid
NULL	NULL	NULL	NULL	NULL

2. View 1: Patient Bill Summary

```
152 •   CREATE OR REPLACE VIEW PatientBillSummary AS
153     SELECT
154         P.PatientID,
155         CONCAT(P.FirstName, ' ', P.LastName) AS PatientName,
156         COUNT(B.BillID) AS NumberOfBills,
157         SUM(B.Amount) AS TotalSpent
158     FROM Patients P
159     JOIN Bills B
160         ON P.PatientID = B.PatientID
161     GROUP BY
162         P.PatientID,
163         P.FirstName,
164         P.LastName;
165
166 •   SELECT * FROM PatientBillSummary;
```

This view summarises the total amount spent by each patient. It is useful for hospital management because it helps identify patients with higher medical expenses and supports financial reporting.

OUTPUT:

PatientID	PatientName	NumberOfBills	TotalSpent
201	James Anderson	1	500.00
202	Sophia Taylor	1	1200.00
203	Daniel Thomas	1	5000.00
204	Olivia Martin	1	300.00
205	William White	1	250.00

View 2: Monthly Appointment Trend

```
172 •   CREATE OR REPLACE VIEW MonthlyAppointmentTrend AS
173     SELECT
174         YEAR(AppointmentDate) AS AppointmentYear,
175         MONTH(AppointmentDate) AS AppointmentMonth,
176         MONTHNAME(AppointmentDate) AS MonthName,
177         COUNT(AppointmentID) AS TotalAppointments
178     FROM Appointments
179     GROUP BY
180         YEAR(AppointmentDate),
181         MONTH(AppointmentDate),
182         MONTHNAME(AppointmentDate)
183     ORDER BY
184         AppointmentYear,
185         AppointmentMonth;
186
187 •   SELECT * FROM MonthlyAppointmentTrend;
```

This view shows appointment demand by month. It helps management plan doctor availability, staffing, and hospital resources.

OUTPUT:

AppointmentYear	AppointmentMonth	MonthName	TotalAppointments
2026	5	May	3
2026	6	June	2

Task 3: Essential SQL Querying and Analytics

Query 1: JOIN and GROUP BY

Which doctor handled the highest number of appointments?

```
1  USE HospitalDB;
2
3  SELECT
4      D.DoctorID,
5      D.DoctorName,
6      D.Specialization,
7      COUNT(A.AppointmentID) AS TotalAppointments
8  FROM Doctors D
9  JOIN Appointments A
10     ON D.DoctorID = A.DoctorID
11 GROUP BY
12     D.DoctorID,
13     D.DoctorName,
14     D.Specialization
15 ORDER BY TotalAppointments DESC;
```

Above SQL command will provide you with the count of appointment made by each individual doctor. In the above SQL command, doctors and appointments tables have been joined together using the column 'DoctorID', that is common between Doctors and Appointments tables. Above SQL command uses the COUNT() function to get count of appointments made per each individual doctor.

The management will be able to see which doctors have the most work. Should one doctor have more patients than other doctors, then this might suggest some reallocations of appointments.

OUTPUT:

DoctorID	DoctorName	Specialization	TotalAppointments
101	Dr. John Smith	Cardiologist	2
102	Dr. Sarah Brown	Neurologist	1
103	Dr. Michael Lee	Orthopedic Surgeon	1
104	Dr. Emma Wilson	Pediatrician	1

Query 2: CASE Classification

How can patients be classified based on their total medical spending?

```
21 •  SELECT
22        P.PatientID,
23        CONCAT(P.FirstName, ' ', P.LastName) AS PatientName,
24        SUM(B.Amount) AS TotalSpent,
25        CASE
26            WHEN SUM(B.Amount) >= 3000 THEN 'High Spender'
27            WHEN SUM(B.Amount) >= 1000 THEN 'Medium Spender'
28            ELSE 'Low Spender'
29        END AS SpendingCategory
30    FROM Patients P
31    JOIN Bills B
32        ON P.PatientID = B.PatientID
33    GROUP BY
34        P.PatientID,
35        P.FirstName,
36        P.LastName
37    ORDER BY TotalSpent DESC;
```

This query connects the table "Patients" and "Bills," calculates the total amount spent by each patient, and classifies patients into high, medium, or low spenders using the CASE function.

This information can be helpful because high spenders could have undergone surgery or some sort of scan done at an additional cost.

OUTPUT:

PatientID	PatientName	TotalSpent	SpendingCategory
203	Daniel Thomas	5000.00	High Spender
202	Sophia Taylor	1200.00	Medium Spender
201	James Anderson	500.00	Low Spender
204	Olivia Martin	300.00	Low Spender
205	William White	250.00	Low Spender

Query 3: Window Function

Which doctors have the highest appointment workload?

```
SELECT
    DoctorID,
    DoctorName,
    Specialization,
    TotalAppointments,
    RANK() OVER (ORDER BY TotalAppointments DESC) AS WorkloadRank
FROM (
    SELECT
        D.DoctorID,
        D.DoctorName,
        D.Specialization,
        COUNT(A.AppointmentID) AS TotalAppointments
    FROM Doctors D
    JOIN Appointments A
        ON D.DoctorID = A.DoctorID
    GROUP BY
        D.DoctorID,
        D.DoctorName,
        D.Specialization
) AS DoctorAppointmentSummary;
```

In this case, the query is utilizing the concept of windowing functions to rank doctors according to the total number of appointments handled by them. In the first step, an inner query is run that combines the tables of Doctors and Appointments through the common field of DoctorID and counts the number of appointments per Doctor.

OUTPUT:

DoctorID	DoctorName	Specialization	TotalAppointments	WorkloadRank
101	Dr. John Smith	Cardiologist	2	1
102	Dr. Sarah Brown	Neurologist	1	2
103	Dr. Michael Lee	Orthopedic Surgeon	1	2
104	Dr. Emma Wilson	Pediatrician	1	2

Task 4: CAP Theorem

1. What is CAP?

CAP Theorem identifies a significant trade-off existing in distributed databases. It says that it's impossible for a distributed system to ensure full Consistency, Availability, and Partition Tolerance simultaneously in case of network failures. These factors are crucial in case there are several servers or places where the data is saved.

Consistency implies that each user can see the most recent version of data. For instance, if a patient's bill gets changed to paid, everyone working in the hospital should see it as paid. Availability assumes that the system will continue to be available regardless of network faults within it. For instance, even in case the network is experiencing difficulties, the doctors as well as receptionists should be able to access patient and appointment records.

2.Why You Cannot Have All Three

But in a distributed system, it is impossible to ensure all three properties of the CAP theorem simultaneously since sometimes there could be a case of network problems causing servers to become separated. This problem is called network partition. And since servers are unable to communicate, it becomes difficult to decide whether the system should strive to maintain data consistency or respond immediately to requests.

In case the system aims at being consistent, then it has to stop serving any requests that could disrupt the integrity of the data in question by not being able to reach an agreement between all servers on what the data means. On the other hand, if availability is a priority, then user requests will always be answered

regardless of the data being outdated at times. It should be noted that network problems do happen in real life.

3. Applying CAP to the Hospital Management System

In a Hospital Management System, the most critical properties of CAP would be Consistency and Partition Tolerance. This application is used to store very crucial data relating to patient information like patient records, appointment details, history of treatment, doctor information, and billing status. When the data stored in such a database turns out to be inconsistent, doctors and other workers might take decisions on the basis of false or outdated information. For instance, while a particular server indicates that the patient has been cured, the other one might not show the updated information.

This highlights the necessity for consistency since all the hospital employees need to have access to consistent data so that no mistakes are made while rendering any kind of service. A doctor will need to have consistent and accurate information about the treatment process to make any clinical decision; similarly, appointment details cannot be double-booked, and there cannot be a delay due to erroneous billing details.

Another reason why Partition Tolerance should not be sacrificed is because hospital management systems can be applied in different departments, buildings, branches, or cloud servers. There could be network failures which might prevent communication between servers. While there is a network failure, the system should focus on the correctness of medical and billing data rather than allowing inconsistencies in the database records. Therefore, in a case of a network partition, it is recommended that the hospital management system continues running despite the network issue.

Since the Hospital Management System prefers Consistency and Partition Tolerance, it will have to sacrifice Availability to some extent during network

issues. Some of the operations would not be available until they can verify that the medical/billing data is correct. The availability might be limited but, in the case of medical data, it would not be wise to allow any inconsistencies to occur due to its importance in a person's life. Therefore, it is better to choose Consistency + Partition Tolerance as the CAP theory option.

Task 5: Query Performance Improvement

A slow or messy query can be created using a nested subquery to calculate each patient's total bill amount.

```
5  SELECT
6      P.PatientID,
7      CONCAT(P.FirstName, ' ', P.LastName) AS PatientName,
8      (
9          SELECT SUM(B.Amount)
10         FROM Bills B
11         WHERE B.PatientID = P.PatientID
12     ) AS TotalSpent
13 FROM Patients P;
```

This query is ok, but it is less efficient because the subquery checks the Bills table for each patient. If the database grows, this type of query may become slow.

PatientID	PatientName	TotalSpent
201	James Anderson	500.00
202	Sophia Taylor	1200.00
203	Daniel Thomas	5000.00
204	Olivia Martin	300.00
205	William White	250.00

The query can be improved using a "JOIN" and "GROUP BY".

```sql
SELECT
    P.PatientID,
    CONCAT(P.FirstName, ' ', P.LastName) AS PatientName,
    SUM(B.Amount) AS TotalSpent
FROM Patients P
JOIN Bills B
    ON P.PatientID = B.PatientID
GROUP BY
    P.PatientID,
    P.FirstName,
    P.LastName;
```

An index can also be added to improve lookup speed:

CREATE INDEX idx_bills_patientid

ON Bills(PatientID);

To check the index:

SHOW INDEX FROM Bills;

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
bills	0	PRIMARY	1	BillID	A	1	NULL	NULL		BTREE			YES	NULL
bills	1	idx_bills_patientid	1	PatientID	A	5	NULL	NULL	YES	BTREE			YES	NULL

This enhanced SQL query is easier to read and more efficient since it directly relates patients to their bills and groups the results. An index can be created on the Bills. PatientID field to make it faster for MySQL to retrieve bills for patients from the table rather than scanning through the entire table. While in smaller databases there won't be a noticeable change in speed, in larger databases, it makes a significant difference.

CONCLUSION

In this report, a relational database model for Hospital Management System has been created with the help of MySQL. There are seven tables that exist within the database such as departments, doctors, patients, appointments, treatments, patient treatment, and bills. For unique record identification, the primary keys have been used whereas foreign keys have been used for establishing the relationship between tables. The inclusion of patient treatment table in the report is required to deal with the many-to-many relationship between patients and treatments.

From the report, one could find the major steps involved in designing the relational database, which include table formation, insertion of some sample data, creating ER diagram, and normalization up to Third Normal Form (3NF).

For the hospital system, two SQL views were generated. First is the PatientBillSummary view, which allows summarizing data related to patient bills. Second, the MonthlyAppointmentTrend view will help managers analyze trends in appointment demand in various months. Also, three analytical SQL queries were prepared: for detecting doctor workloads using JOIN and GROUP BY, for classifying patients based on the amount of money they spend using CASE, and for ranking doctors based on appointments workloads using RANK() window function.

Using CAP theorem for the Hospital Management System, it can be said that consistency and partition tolerance are the most crucial properties in this database because the data is sensitive. Additionally, query performance was analyzed as well; a nested subquery was replaced with JOIN and GROUP BY queries and also an index on Bills(PatientID) was added.

Generally, the Hospital Management System database is an illustration of the application of relational database design, SQL programming, creation of views, analytical queries, database normalization, CAP theorem, and query optimization in real-life situations.

BIBLIOGRAPHY

References

Brewer, E. (2012) 'CAP twelve years later: How the "rules" have changed', Computer, 45(2), pp. 23–29.

Connolly, T. and Begg, C. (2015) Database Systems: A Practical Approach to Design, Implementation, and Management. 6th edn. Harlow: Pearson Education.

Coronel, C. and Morris, S. (2019) Database Systems: Design, Implementation, & Management. 13th edn. Boston: Cengage Learning.

Date, C.J. (2004) An Introduction to Database Systems. 8th edn. Boston: Pearson/Addison Wesley.

Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. 7th edn. Boston: Pearson.

Kleppmann, M. (2017) Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media.

Oracle (2026) MySQL 8.4 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.4/en/ (Accessed: 16 June 2026).

Silberschatz, A., Korth, H.F. and Sudarshan, S. (2020) Database System Concepts. 7th edn. New York: McGraw-Hill Education.